

Time Series Analysis

Havana Rika



Ben-Gurion University
of the Negev

EDA for TS Data

We will use the following libraries:

- ❖ scipy
- ❖ numpy
- ❖ pandas
- ❖ seaborn
- ❖ scikit-learn

Datetime

To get a date, we can do this:

```
from datetime import date
```

To get today's date:

```
today = date.today()
```

To get some other date:

```
other_date = date(2021, 3, 24)
```

If we want a datetime object (a timestamp) instead, we can do this as well:

```
from datetime import datetime  
now = datetime.now()
```

This will get the current timestamp. We can create a datetime for a specific date and time as well:

```
some_date = datetime(2021, 5, 18, 15, 39, 0)  
some_date.isoformat()
```

We can get a string output in isoformat:

```
'2021-05-18T15:39:00'
```

Datetime

We can also work with time differences using timedelta:

```
from datetime import timedelta  
year = timedelta(days=365)
```

These timedelta objects can be added to other objects for calculations. We can do calculations with a timedelta object, for example:

```
year * 10
```

This should give us the following output:

```
datetime.timedelta(days=3650)
```

```
d1 = datetime(2025, 1, 1)  
d2 = datetime(2025, 1, 10)  
difference = d2 - d1 # Returns a timedelta of 9 days
```

Datetime

The datetime library can parse string inputs to date and datetime types and output these objects as string:

```
from datetime import date
some_date = date.fromisoformat('2021-03-24')
```

Or:

```
some_date = datetime.date(2021, 3, 24)
```

We can format the output with string format options, for example like this:

```
some_date.strftime('%A %d. %B %Y')
```

This would give us:

```
'Wednesday 24. March 2021'
```

Similarly, we can read in a date or datetime object from a string, and we can use the same format options:

```
from datetime import datetime
dt = datetime.strptime('24/03/21 15:48', '%d/%m/%y %H:%M')
```

```
from datetime import datetime

# Standard ISO format (YYYY-MM-DD HH:MM:SS)
dt = datetime.fromisoformat('2025-07-21 10:30:45')
print(dt) # Output: 2025-07-21 10:30:45
```

Format string	Meaning
%Y	Year as 4 digits
%y	Year as 2 digits
%m	Month as a number
%d	Day
%H	Hour as 2 digits
%M	Minute as 2 digits

<https://strftime.org/>

pandas

```
import pandas as pd
pd.date_range(start='2021-03-24', end='2021-09-01')
```

This gives us a `DatetimeIndex` like this:

```
DatetimeIndex(['2021-03-24', '2021-03-25', '2021-03-26', '2021-03-27',
               '2021-03-28', '2021-03-29', '2021-03-30', '2021-03-31',
               '2021-04-01', '2021-04-02',
               ...,
               '2021-08-23', '2021-08-24', '2021-08-25', '2021-08-26',
               '2021-08-27', '2021-08-28', '2021-08-29', '2021-08-30',
               '2021-08-31', '2021-09-01'],
              dtype='datetime64[ns]', length=162, freq='D')
```

We can also create a time-series as follows:

```
pd.Series(pd.date_range("2021", freq="D", periods=3))
```

This would give us a time-series like this:

```
0    2021-01-01
1    2021-01-02
2    2021-01-03
dtype: datetime64[ns]
```

One of the most important functionalities is parsing to date or datetime objects from either string or separate columns:

```
import pandas as pd
df = pd.DataFrame({'year': [2021, 2022],
                  'month': [3, 4],
                  'day': [24, 25]})

ts1 = pd.to_datetime(df)
ts2 = pd.to_datetime('20210324', format='%Y%m%d')
```

```
0    2021-03-24
1    2022-04-25
dtype: datetime64[ns]
```

```
Timestamp('2021-03-24 00:00:00')
```

You can take a rolling window for calculations like this:

```
s = pd.Series([1, 2, 3, 4, 5])
s.rolling(3).sum()
```

```
0    NaN
1    NaN
2    6.0
3    9.0
4   12.0
dtype: float64
```

pandas

A time-series would usually be an index with a time object and one or more columns with numeric or other types, such as this:

```
import numpy as np
rng = pd.date_range('2021-03-24', '2021-09-01', freq='D')
ts = pd.Series(np.random.randn(len(rng)), index=rng)
```

We can have a look at our time-series:

```
2021-03-24    -2.332713
2021-03-25     0.177074
2021-03-26    -2.136295
2021-03-27     2.992240
2021-03-28    -0.457537
...
2021-08-28    -0.705022
2021-08-29     1.089697
2021-08-30     0.384947
2021-08-31     1.003391
2021-09-01    -1.021058
Freq: D, Length: 162, dtype: float64
```

pandas

We can index these time-series datasets like any other pandas Series or DataFrame. `ts[:2].index` would give us:

```
DatetimeIndex(['2021-03-24', '2021-03-25'], dtype='datetime64[ns]',  
              freq='D')
```

Interestingly, we can index directly with strings or datetime objects. For example, `ts['2021-03-28':'2021-03-30']` gives us:

```
2021-03-28    -0.457537  
2021-03-29    -1.089423  
2021-03-30    -0.708091  
Freq: D, dtype: float64
```


pandas

You can shift or lag the values in a time-series back and forward in time using the `shift` method. This changes the alignment of the data:

```
ts.shift(1)[:5]
```

We can also change the resolution of time-series objects, for example like this:

```
ts.asfreq('M')
```

```
ts.shift(1)
```

Result:

2021-03-24	10
2021-03-25	20
2021-03-26	30
2021-03-27	40

Freq: D

2021-03-24	NaN
2021-03-25	10.0
2021-03-26	20.0
2021-03-27	30.0

Freq: D

`shift(1)` vs `shift(-1)`

Suppose we have:

2021-03-24	10
2021-03-26	30
2021-03-27	40

Notice that `2021-03-25` is missing.

```
ts.asfreq("D")
```

Result:

2021-03-24	10.0
2021-03-25	NaN
2021-03-26	30.0
2021-03-27	40.0

Freq: D

pandas

You can shift or lag the values in a time-series back and forward in time using the `shift` method. This changes the alignment of the data:

```
ts.shift(1)[:5]
```

We can also change the resolution of time-series objects, for example like this:

```
ts.asfreq('M')
```

Common frequency codes

Code	Meaning
"D"	Daily
"H"	Hourly
"30min"	Every 30 minutes
"W"	Weekly
"M"	Month end
"MS"	Month start
"Y"	Year end

Understanding the variables

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$CI^+ = \bar{x} + (SE \times 1.96)$$

$$\sigma_X = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} = \sqrt{\mathbb{E}[(X - \bar{x})^2]}$$

$$CI^- = \bar{x} - (SE \times 1.96)$$

$$\text{median}(X) = x_{(n+1)/2}$$

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}}$$

Understanding the variables

```
pollution = pd.read_csv(  
    'https://raw.githubusercontent.com/owid/owid-datasets/master/  
datasets/Air%20pollution%20by%20city%20-%20Fouquet%20and%20DPCC%20  
(2011)/Air%20pollution%20by%20city%20-%20Fouquet%20and%20DPCC%20(2011).  
csv'  
)  
len(pollution)
```

```
331
```

```
pollution.columns
```

```
Index(['Entity', 'Year', 'Smoke (Fouquet and DPCC (2011))',  
      'Suspended Particulate Matter (SPM) (Fouquet and DPCC (2011))'],  
      dtype='object')
```

Understanding the variables

```
pollution = pollution.rename(  
    columns={  
        'Suspended Particulate Matter (SPM) (Fouquet and DPCC (2011))':  
        'SPM',  
        'Smoke (Fouquet and DPCC (2011))' : 'Smoke',  
        'Entity': 'City'  
    }  
)  
pollution.dtypes
```

Here's the output:

```
City                object  
Year                int64  
Smoke              float64  
SPM                float64  
dtype: object
```

```
pollution.City.unique()
```

```
array(['Delhi', 'London'], dtype=object)
```

```
pollution.Year.min(), pollution.Year.max()
```

The minimum and the maximum year are these:

```
(1700, 2016)
```

Understanding the variables

We'll convert our Year column from int64 to datetime. This will help with plotting:

```
pollution['Year'] = pollution['Year'].apply(
    lambda x: datetime.datetime.strptime(str(x), '%Y')
)
pollution.dtypes
```

```
City      object
Year      datetime64[ns]
Smoke     float64
SPM       float64
dtype: object
```

Let's check for missing values and get descriptive summary statistics of columns:

```
pollution.isnull().mean()
```

```
City      0.000000
Year      0.000000
Smoke     0.090634
SPM       0.000000
dtype: float64
```

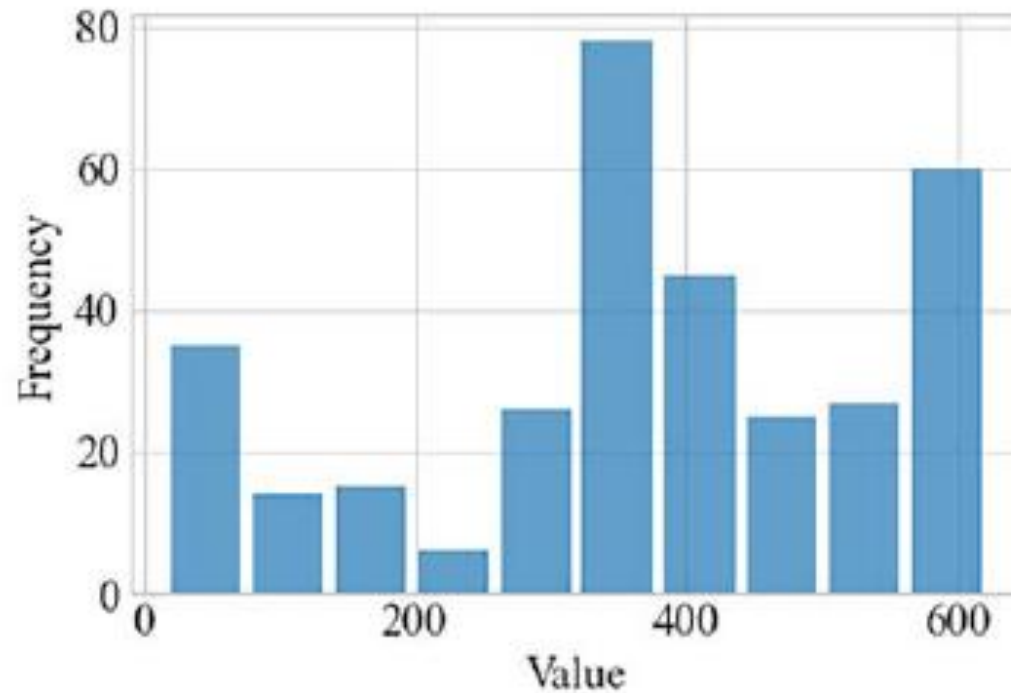
```
pollution.describe()
```

	Smoke	SPM
count	301.000000	331.000000
mean	210.296440	365.970050
std	88.543288	172.512674
min	13.750000	15.000000
25%	168.571429	288.474026
50%	208.214286	375.324675
75%	291.818182	512.609209
max	342.857143	623.376623

Understanding the variables

Let's plot a histogram of the SPM variable:

```
n, bins, patches = plt.hist(  
    x=pollution['SPM'], bins='auto',  
    alpha=0.7, rwidth=0.85  
)  
plt.grid(axis='y', alpha=0.75)  
plt.xlabel('SPM')  
plt.ylabel('Frequency')
```



Uncovering relationships between variables

The **relationship between the variables needs to be investigated**. This includes the direction and rough size of any correlations to avoid feature leakage and collinearity.

Feature leakage is when a variable unintentionally gives away the target

Collinearity means that independent variables (features) are correlated. The latter case can be problematic in linear models. Therefore, if we carry out linear regression and find two variables that are highly correlated between themselves, we should remove one of them or use dimensionality reduction techniques such as Principal Component Analysis (PCA).

Linear relationships between variables

The **Pearson correlation coefficient** was developed by Karl Pearson, whom we've discussed in the previous chapter, and named in his honor to distinguish it from other correlation coefficients. The Pearson correlation coefficient between two variables X and Y is defined as follows:

$$\rho_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y}$$

$\text{cov}(X, Y)$ is the covariance between the two variables defined as the expected value (the mean) between the differences of each point to the variable mean:

$$\text{cov}(X, Y) = \mathbb{E}[(X - \bar{x})(Y - \bar{y})]$$

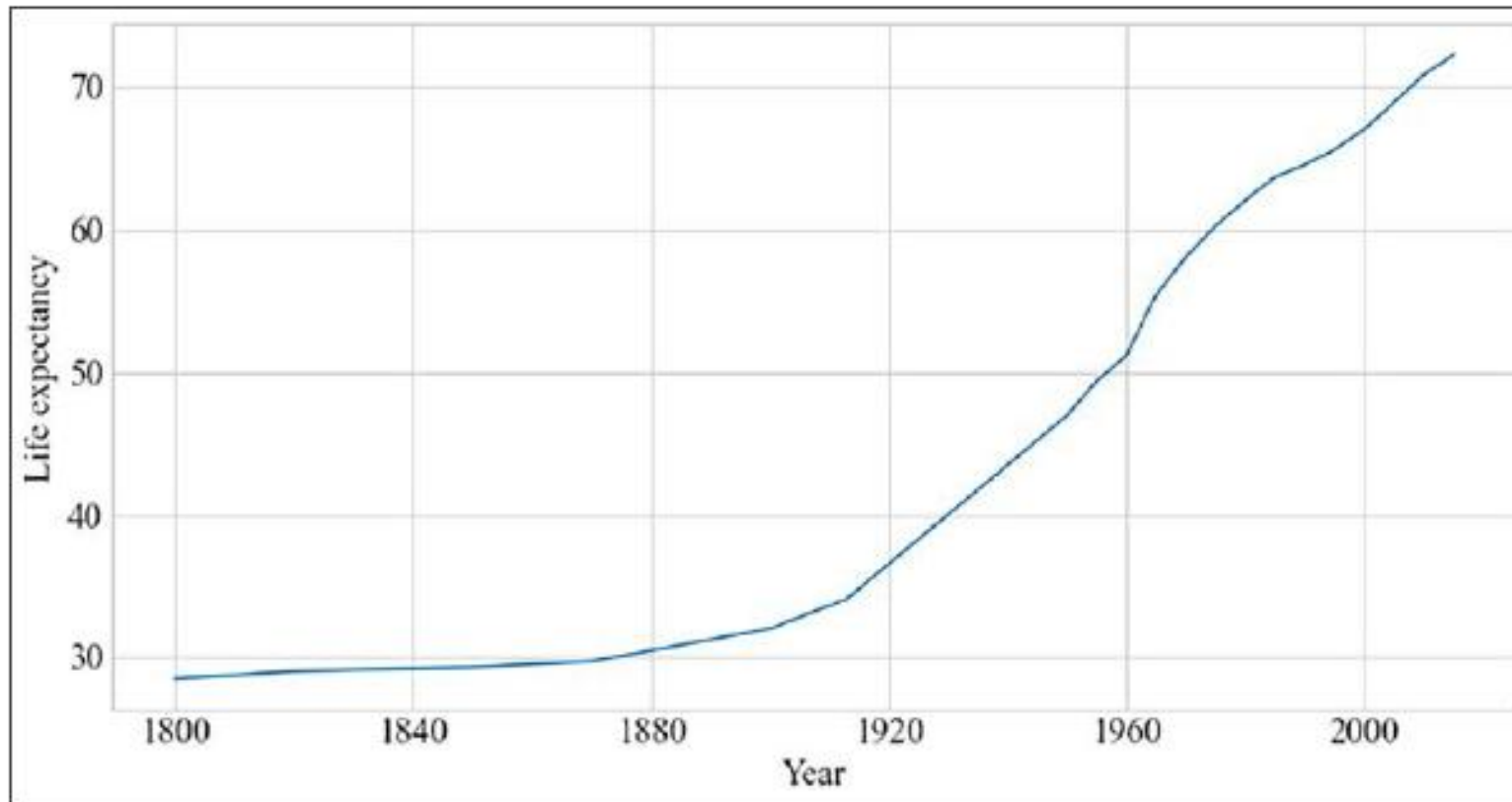
σ_X is the standard deviation of the variable X .

Expanded, the formula looks like this:

$$\hat{\rho}_{X,Y} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sigma_x \sigma_y}$$

Linear relationships between variables

If we looked at a plot of global life expectancy from 1800 onward, we'd see an increase of years lived with the time axis. You can see the plot of global life expectancy based on data on OWID:



Linear relationships between variables

In order to calculate the Pearson correlation, we can use a function from SciPy:

```
from scipy import stats

def ignore_nans(a, b):
    index = ~a.isnull() & ~b.isnull()
    return a[index], b[index]

stats.pearsonr(*ignore_nans(pollution['Smoke'], pollution['SPM']))
```

Here's the Pearson correlation and the p-value that indicates significance (the lower, the more significant)

```
(0.9454809183096181, 3.313283689287137e-10)
```

Linear relationships between variables

Conversely, we would see a negative correlation of time with child mortality – as the year increases, child mortality decreases. This plot shows the child mortality per 1,000 children on data taken from OWID:

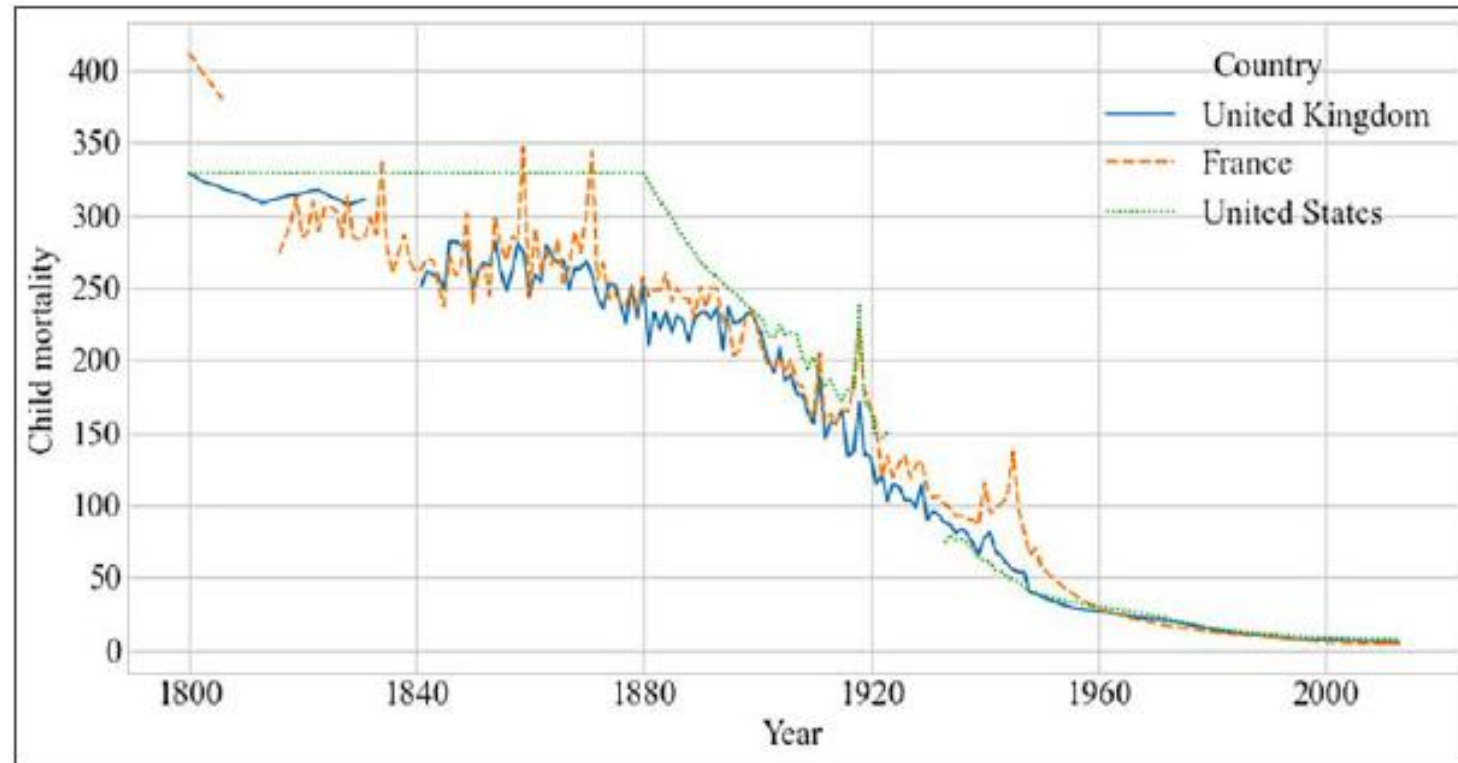


Figure 2.5: Child mortality from 1800 to today in the UK, France, and the USA

Linear relationships between variables



Figure 2.6: Correlation heatmap of child mortality between the UK, France, and the USA

In this correlation heatmap, we can see that countries are highly correlated (for example, a correlation of 0.78 between France and the United Kingdom).

Linear relationships between variables

The correlation coefficient struggles with cases where the increases are **non-linear** or **non-continuous**, or (because of the squared term) when there are outliers.

A **scatter plot** can be used for showing and comparing numeric values. It plots values of two variables against each other. Usually, the variables are numerical – otherwise, we'd call this a table.

We can **combine** a **scatter plot** with the **histograms** of the variables we are plotting against each other, so we can see how many points on one or the other variable are being displayed.

Scatter plots often have a **best-fit line** superimposed in order to visualize how one variable is the function of another variable.

Linear relationships between variables

Here's an example of how to plot a scatter plot with marginal histograms of the two variables in the pollution dataset:

```
plt.figure(figsize=(12, 6))
sns.jointplot(
    x="Smoke", y="SPM",
    edgecolor="white",
    data=pollution
)
plt.xlabel("Smoke")
plt.ylabel("SPM");
```

Here's the resulting plot:

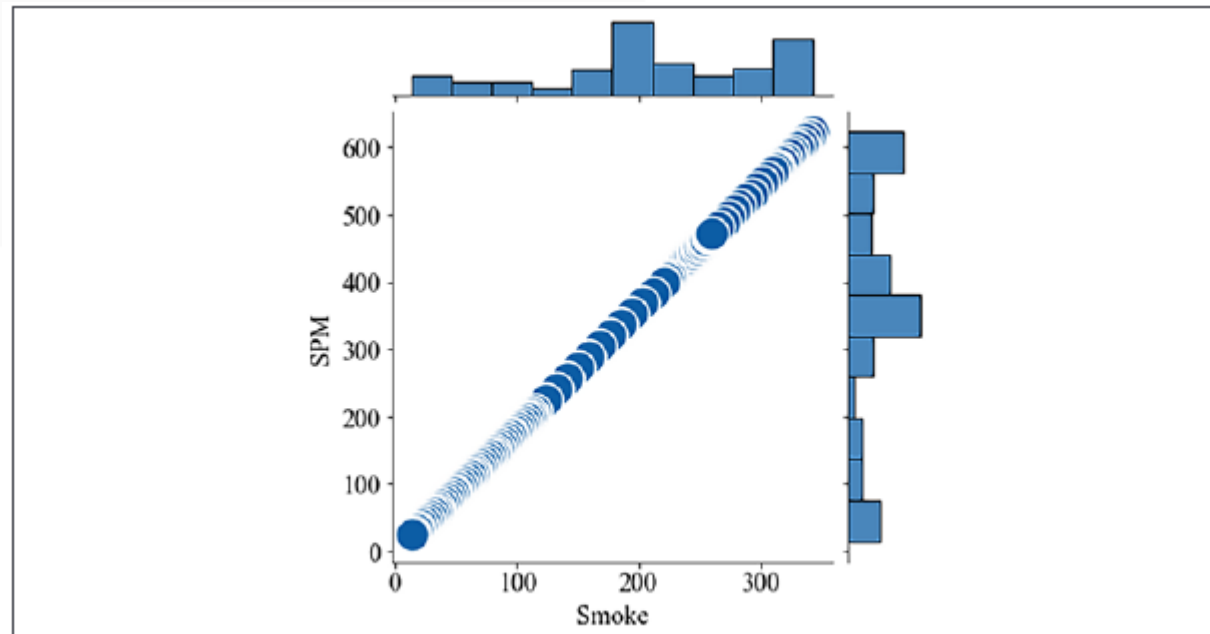
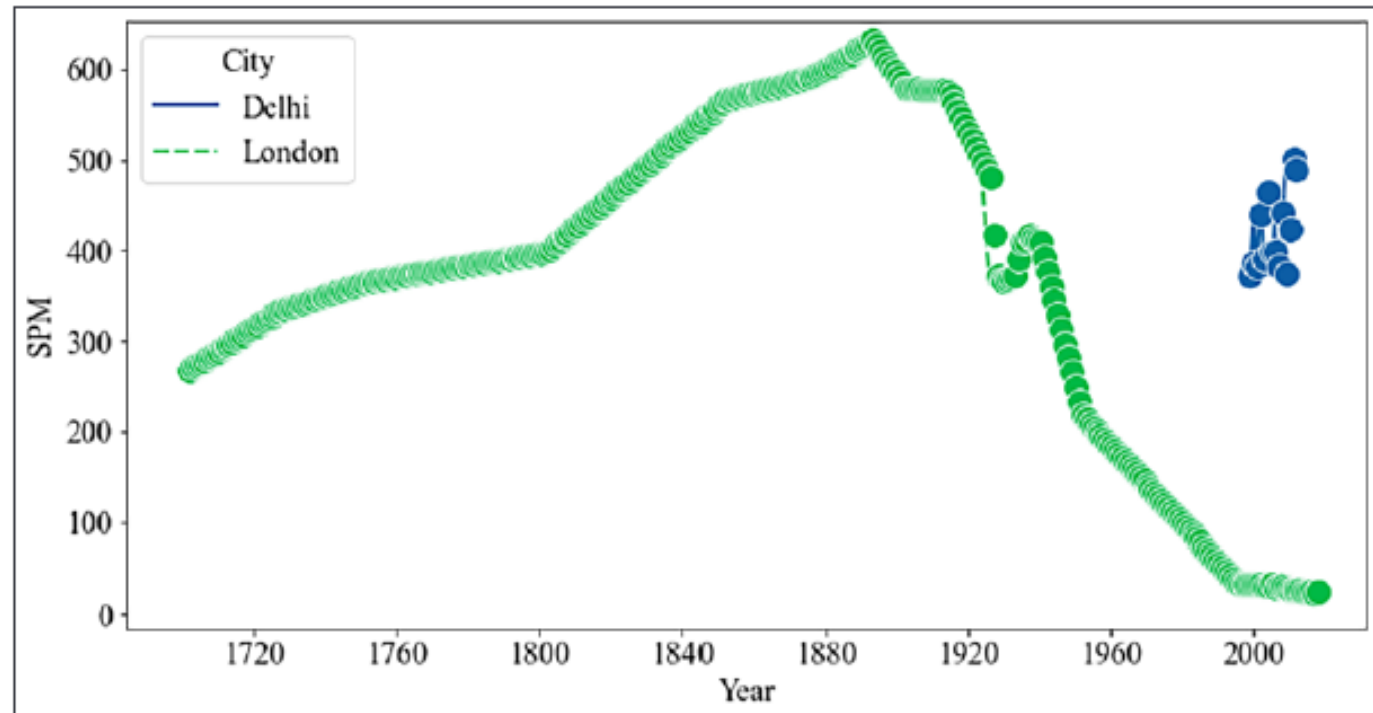


Figure 2.7: Scatter plot with marginal histograms of Smoke against SPM

Linear relationships between variables

We've seen the dataset of Suspended Particulate Matter (SPM) before. Let's plot SPM over time:

```
pollution = pollution.pivot("Year", "City", "SPM")  
plt.figure(figsize=(12, 6))  
sns.lineplot(data=pollution)  
plt.ylabel('SPM');
```



Linear relationships between variables

The **Spearman rank correlation** can **handle outliers** and **non-linear relationships** much better than the Pearson correlation coefficient – although it can't handle non-continuous cases like the one above.

The **Spearman** correlation is the **Pearson** correlation, only applied on **ranks** of variables' values instead of the variables' values directly.

The Spearman correlation of the time-series for **air pollution** is **-0.19**, and for the **two time periods before and after 1880** we get **0.99** and **-0.99**, respectively. In the case of the Spearman correlation coefficient, the numerical differences are ignored – what counts is the order of the points. In this case, the order of the points within the two time periods aligns nearly perfectly.

Identifying trend and seasonality

Trend, seasonality, and cyclic variations are the most important characteristics of time-series.

A **trend** is the presence of a long-term increase or decrease in the sequence.

Seasonality is a variation that occurs at specific regular intervals of less than a year. Seasonality can occur on different time spans such as daily, weekly, monthly, or yearly.

Finally, **cyclic variations** are rises and falls that are not of a fixed frequency.

Identifying trend and seasonality

An important characteristic of time-series is **stationarity**.

This refers to a property of **time-series not to change distribution over time**, or in other words, that **the process that produces the time-series doesn't change with time**.

Many models or measures **assume stationarity** and might not work properly if the data is not stationary.

Therefore, with these algorithms, the time-series should be decomposed first into the **main signal**, and then the **seasonal** and **trend** components.

In this decomposition, we would subtract the trend and seasonal components from the original time-series.

Identifying trend and seasonality

We'll download the dataset from Datahub (<https://datahub.io/core/global-temp>) or you can find it from the book's GitHub repository (in the chapter2 folder).

Then, we can load it up and pivot it:

```
temperatures = pd.read_csv('/Users/ben/Downloads/monthly_csv.csv')
temperatures['Date'] = pd.to_datetime(temperatures['Date'])
temperatures = temperatures.pivot('Date', 'Source', 'Mean')
```

Now we can use NumPy's polyfit functionality. It fits a polynomial of the form:

$$f(x) = \sum_{i=0}^k b_i x^{i+1}$$

In this formula, k is the degree of the polynomial and b is the coefficients we are trying to find.

Identifying trend and seasonality

```
from numpy import polyfit

def fit(X, y, degree=3):
    coef = polyfit(X, y, degree)
    trendpoly = np.poly1d(coef)
    return trendpoly(X)

def get_season(s, yearly_periods=4, degree=3):
    X = [i%(365/4) for i in range(0, len(s))]
    seasonal = fit(X, s.values, degree)
    return pd.Series(data=seasonal, index=s.index)

def get_trend(s, degree=3):
    X = list(range(len(s)))
    trend = fit(X, s.values, degree)
    return pd.Series(data=trend, index=s.index)
```

```
import seaborn as sns
```

```
plt.figure(figsize=(12, 6))
```

```
temperatures['trend'] = get_trend(temperatures['GCAG'])
temperatures['season'] = get_season(temperatures['GCAG'] -
temperatures['trend'])
sns.lineplot(data=temperatures[['GCAG', 'season', 'trend']])
plt.ylabel('Temperature change');
```

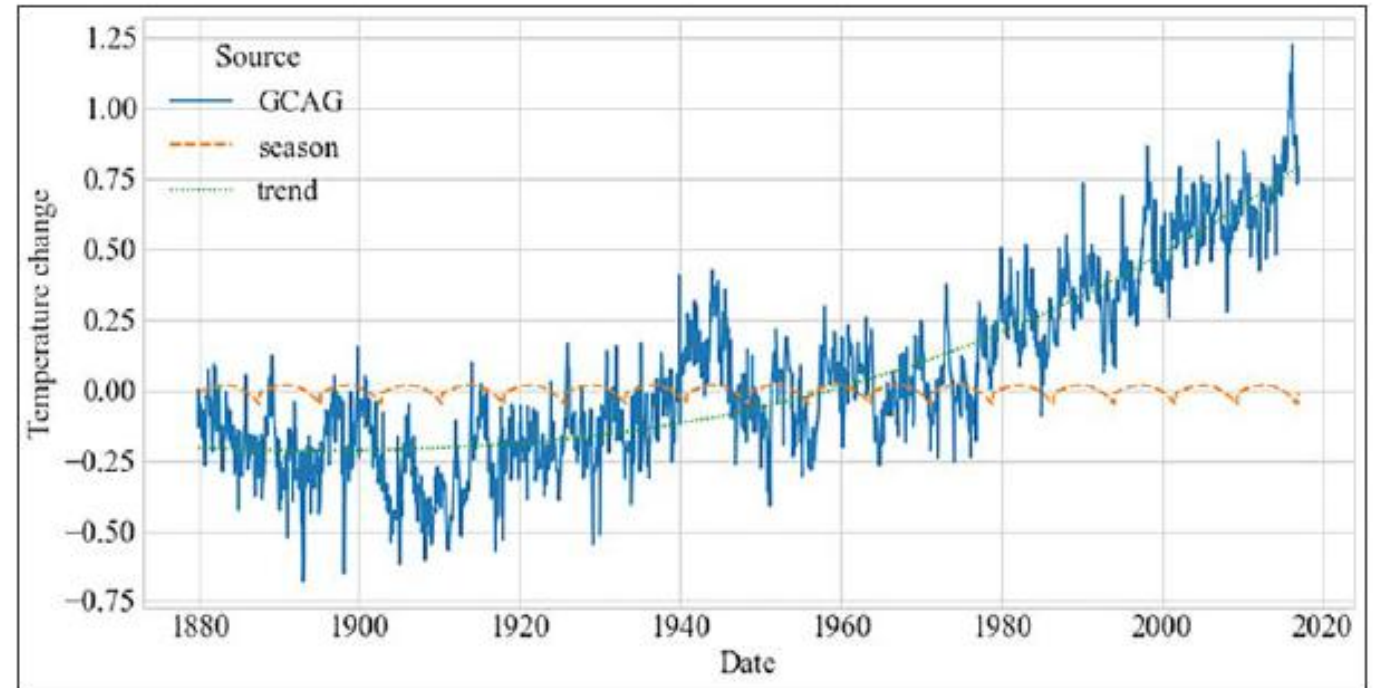


Figure 2.9: Temperature change from the late 19th century to today

Identifying trend and seasonality

Autocorrelation is the correlation of a signal with a lagged version of itself. The autocorrelation plot draws the autocorrelation as a function of lag.

The autocorrelation plot can **help find repeating patterns**, and is often used in signal processing.

The autocorrelation can help spot a periodic signal.

Identifying trend and seasonality

```
pollution = pollution.pivot("Year", "City", "SPM")  
pd.plotting.autocorrelation_plot(pollution['London'])
```

Here's the plot that we get:

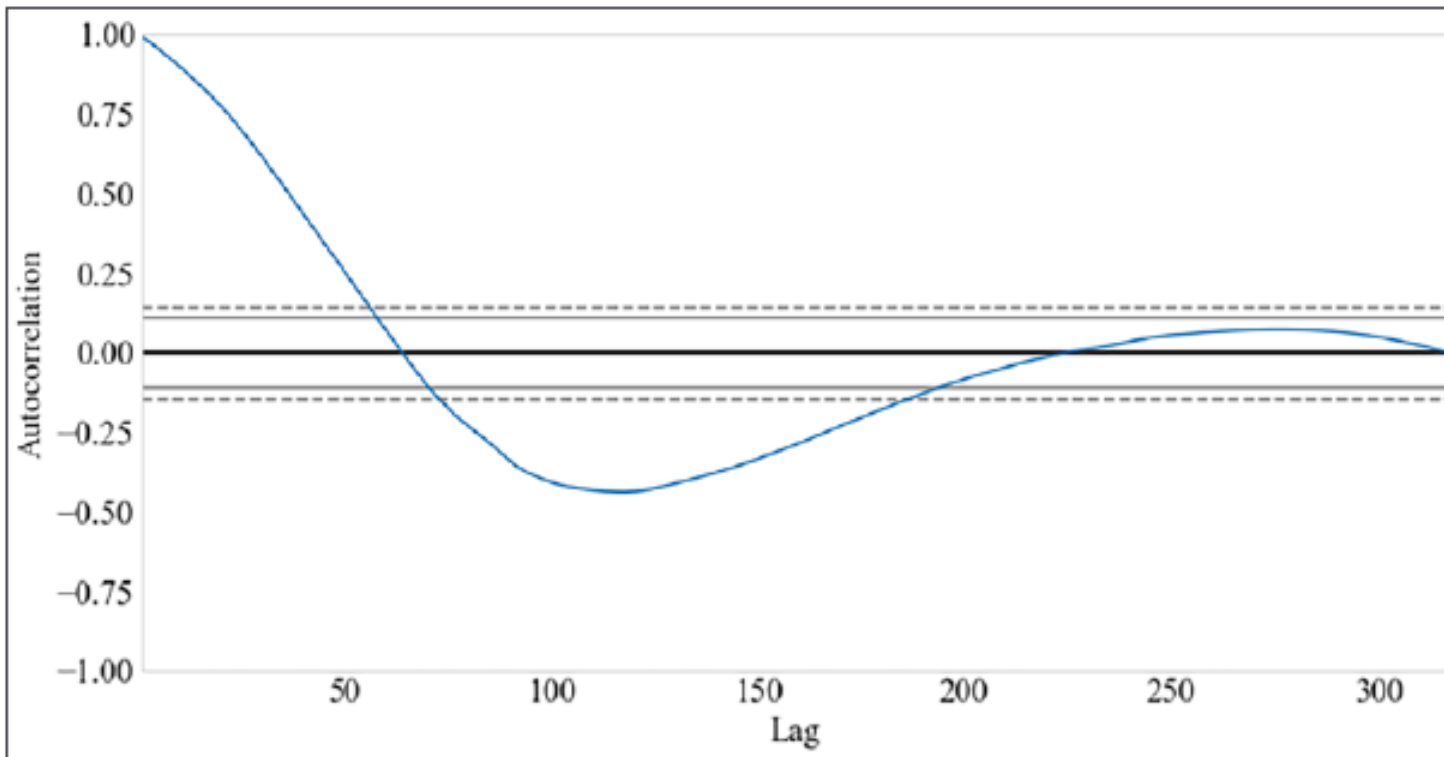


Figure 2.10: Autocorrelation plot of pollution in London

Identifying trend and seasonality

The plot of SPM clearly shows that air pollution is **not** a **stationary** process, since the **autocorrelation** is **not flat**. You can also compare the run of pollution that shows there's a trend, and therefore the mean also changes – another indication that the series is not stationary.

We can also test this statistically. A test for stationarity is the augmented Dickey-Fuller test:

```
from statsmodels.tsa import stattools  
  
stattools.adfuller(pollution['London'])
```

```
(-0.33721640804242853,  
 0.9200654843183897,  
 13,  
 303,  
  
 {'1%': -3.4521175397304784,  
  '5%': -2.8711265007266666,  
  '10%': -2.571877823851692},  
 1684.6992663493872)
```


Identifying trend and seasonality

Electroencephalography (EEG) signals show brain waves at several frequency ranges.

```
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.dates import DateFormatter
import seaborn as sns
from sklearn.datasets import fetch_openml
```

```
eeg = fetch_openml(data_id=1471, as_frame=True)
increment = 117 / len(eeg['data'])
import numpy as np
index = np.linspace(
    start=0,
    stop=increment*len(eeg['data']),
    num=len(eeg['data'])
)
ts_index = pd.to_datetime(index, unit='s')
v1 = pd.Series(name='V1', data=eeg['data']['V1'].values, index=ts_index)
```

```
slicing = (v1.index >= '1970-01-01 00:00:08') & (v1.index <='1970-01-01
00:01:10.000000000')
v1[slicing]
```

Here's the slice:

```
1970-01-01 00:00:08.006208692    4289.74
1970-01-01 00:00:08.014019627    4284.10
1970-01-01 00:00:08.021830563    4280.00
1970-01-01 00:00:08.029641498    4289.74
1970-01-01 00:00:08.037452433    4298.46
...
1970-01-01 00:01:09.962547567    4289.74
1970-01-01 00:01:09.970358502    4283.08
1970-01-01 00:01:09.978169437    4284.62
1970-01-01 00:01:09.985980373    4289.23
1970-01-01 00:01:09.993791308    4290.77
Name: V1, Length: 7937, dtype: float64
```

Identifying trend and seasonality

Electroencephalography (EEG) signals show brain waves at several frequency ranges.

```
date_formatter = DateFormatter("%S")  
ax = v1[slicing].plot(figsize=(12, 6))  
ax.xaxis.set_major_formatter(date_formatter)  
plt.ylabel('voltage')
```

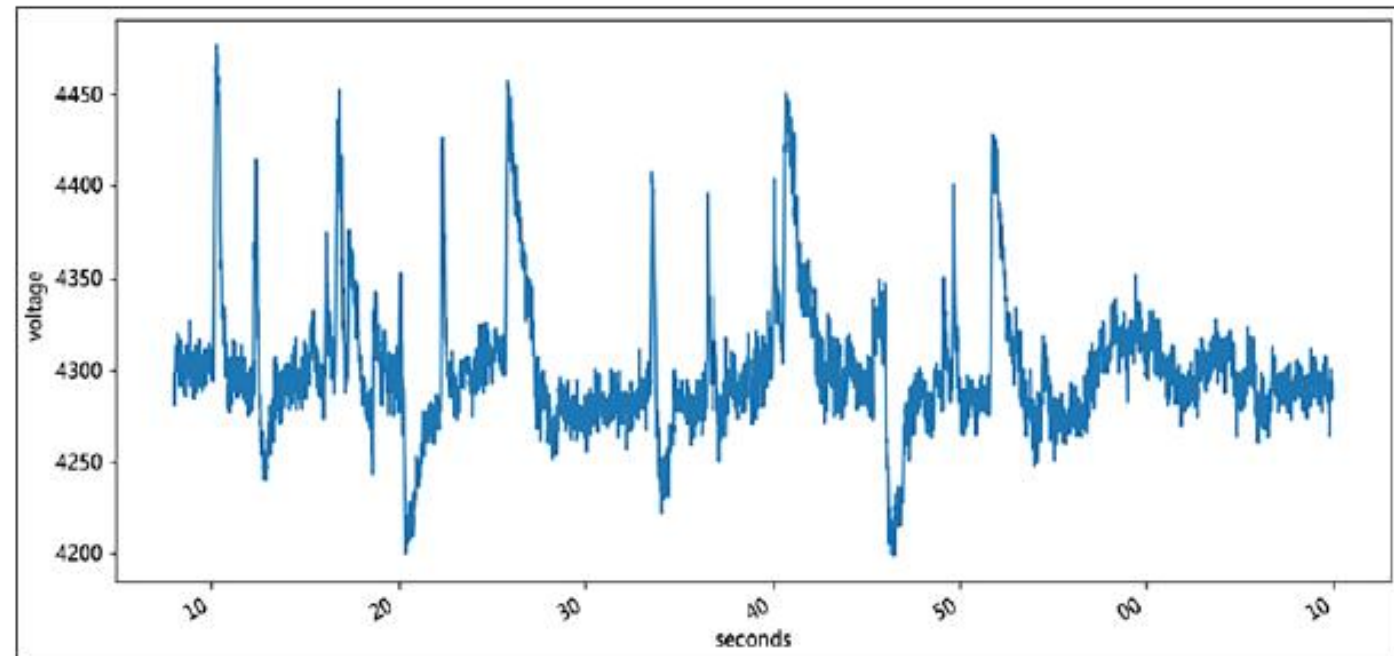


Figure 2.11: Voltage over time in an EEG signal

Resample the Data

```
plt.subplot(311)
ax1 = v1[slicing].resample('1s').mean().plot(figsize=(12, 6))
ax1.xaxis.set_major_formatter(date_formatter)
plt.subplot(312)
ax1 = v1[slicing].resample('2s').mean().plot(figsize=(12, 6))
ax1.xaxis.set_major_formatter(date_formatter)
plt.subplot(313)
ax2 = v1[slicing].resample('5s').mean().plot(figsize=(12, 6))
ax2.xaxis.set_major_formatter(date_formatter)
plt.xlabel('seconds');
```

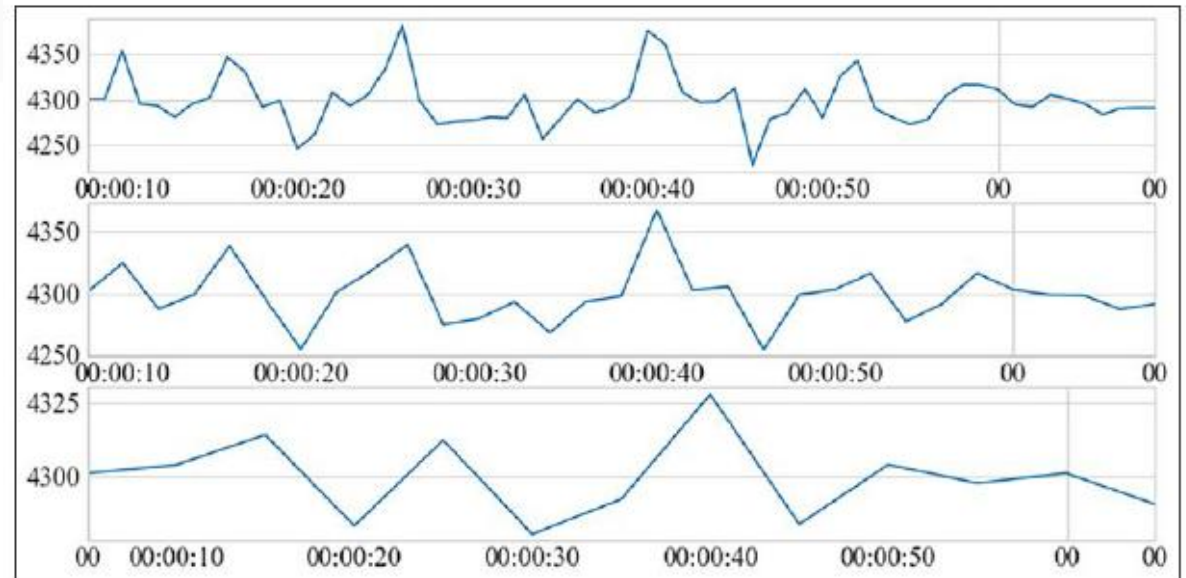


Figure 2.12: Resampled EEG signals

Resample the Data

```
from scipy import signal
fs = len(eeg['data']) // 117
f, Pxx_den = signal.welch(
    v1[slicing].values,
    fs,
    nperseg=2048,
    scaling='spectrum'
)
plt.semilogy(f, Pxx_den)
plt.xlabel('frequency [Hz]')
plt.ylabel('PSD [V**2/Hz]')
```

The spectral density plot, the periodogram, looks like this:

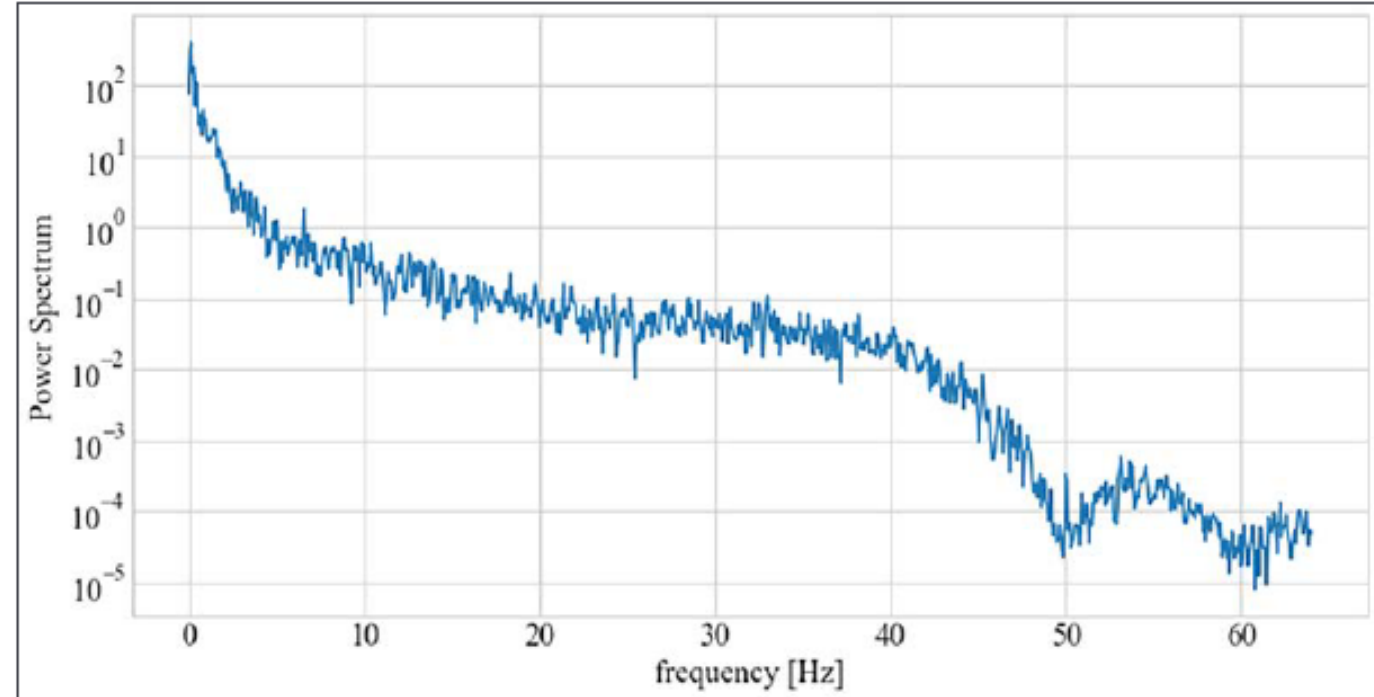


Figure 2.13: Periodogram of the EEG signals

Other Examples

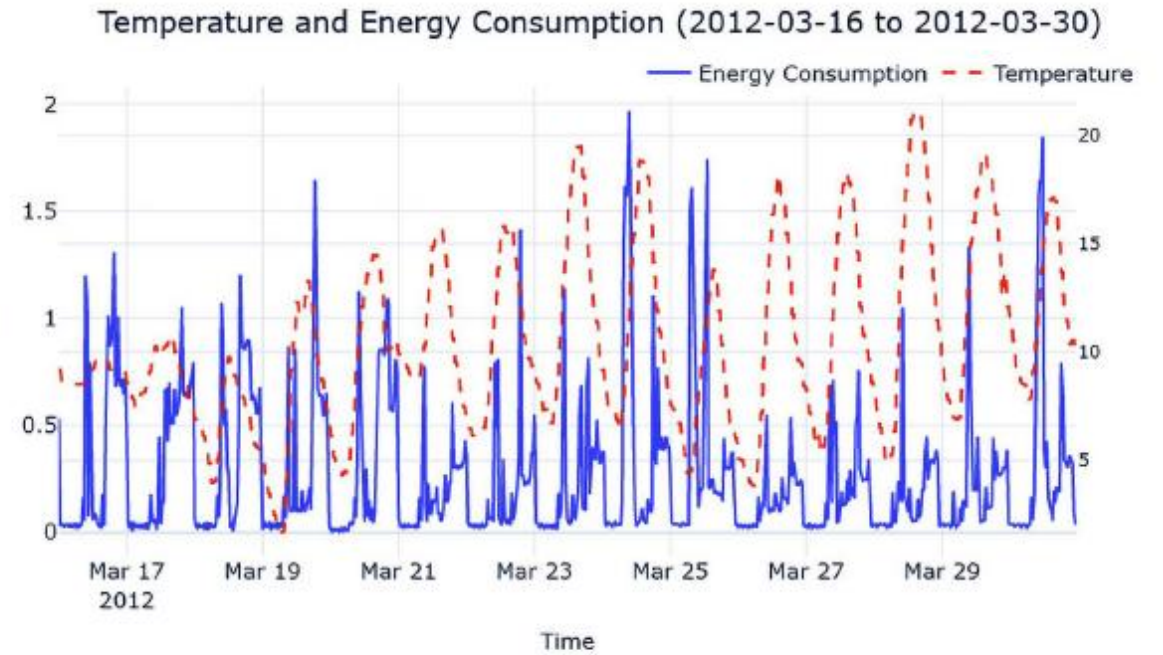
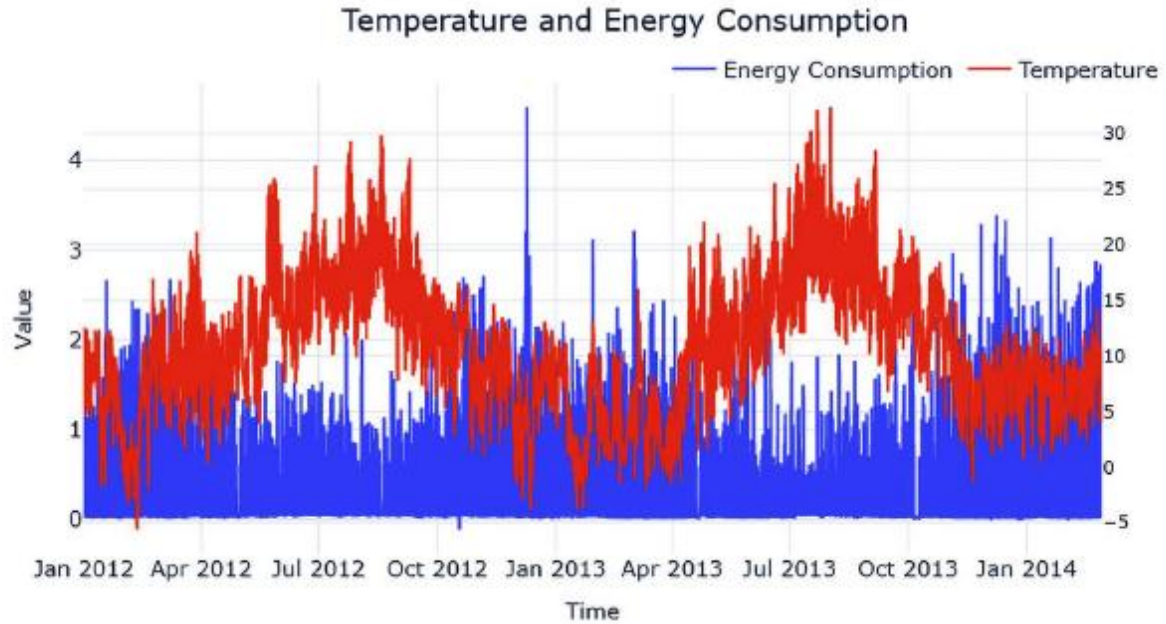


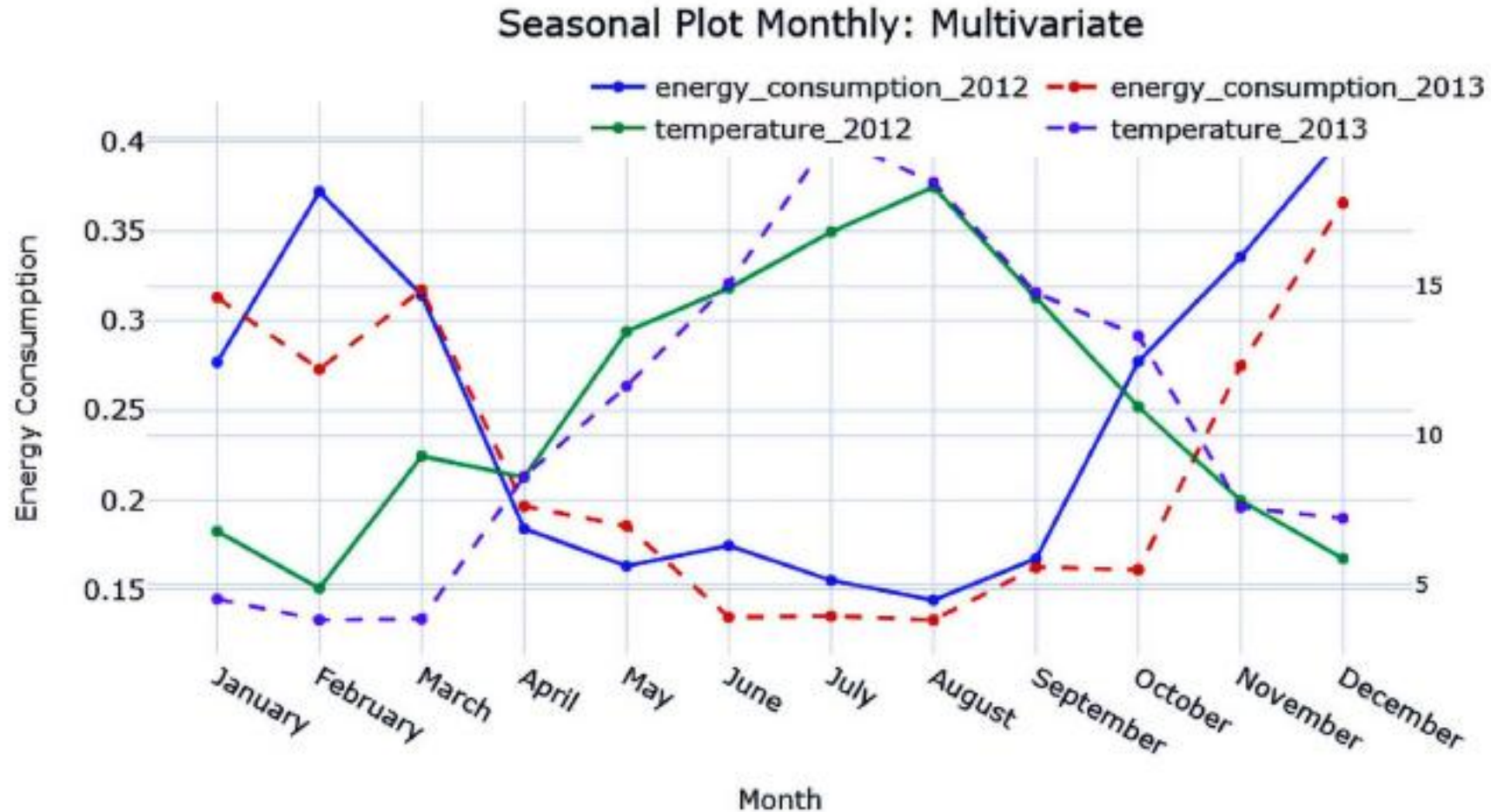
Figure 3.5: Temperature and energy consumption (zoomed-in plot at the bottom)

Other Examples



Figure 3.6: Seasonal plot at a monthly resolution

Other Examples



Seasonal box plots

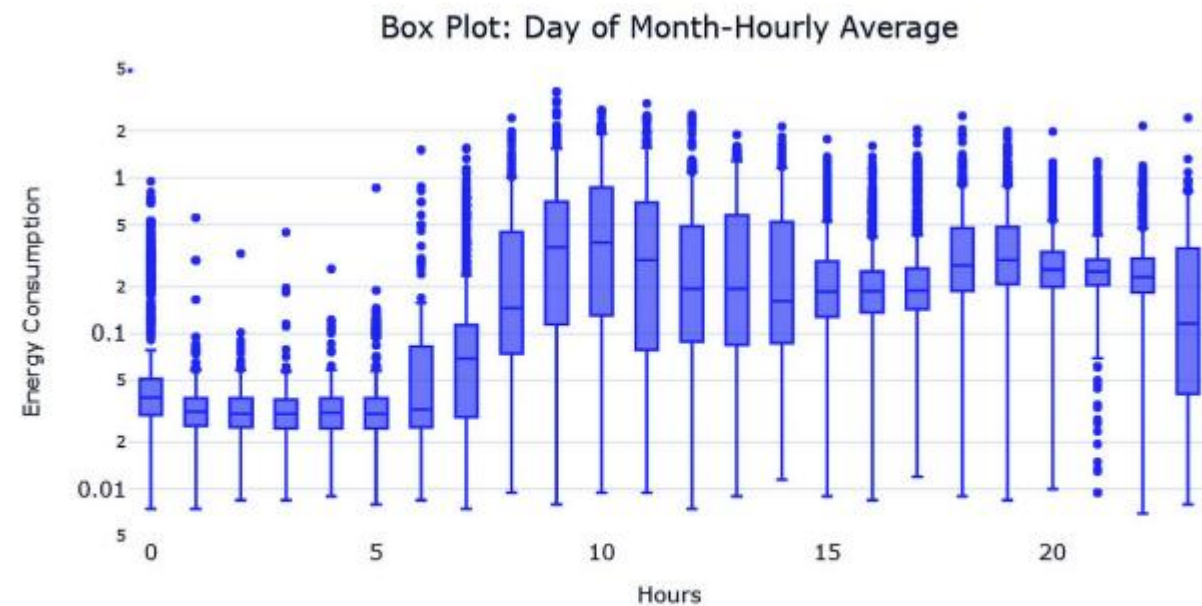
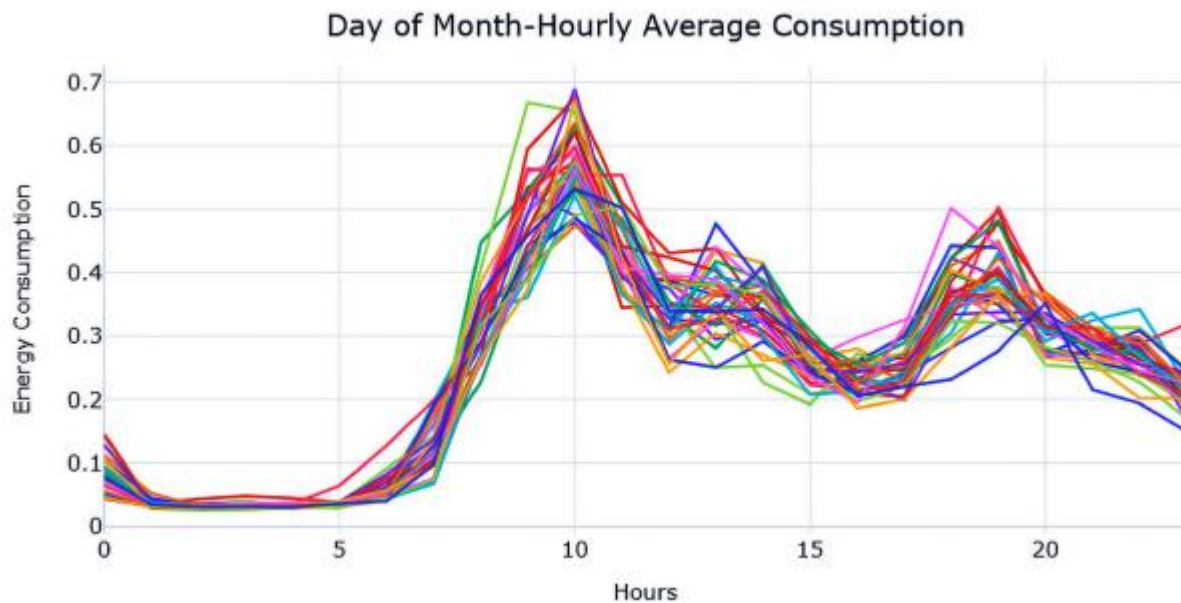


Figure 3.8: Seasonal plot (top) and seasonal box plot (bottom) at an hourly resolution

Calendar heatmaps

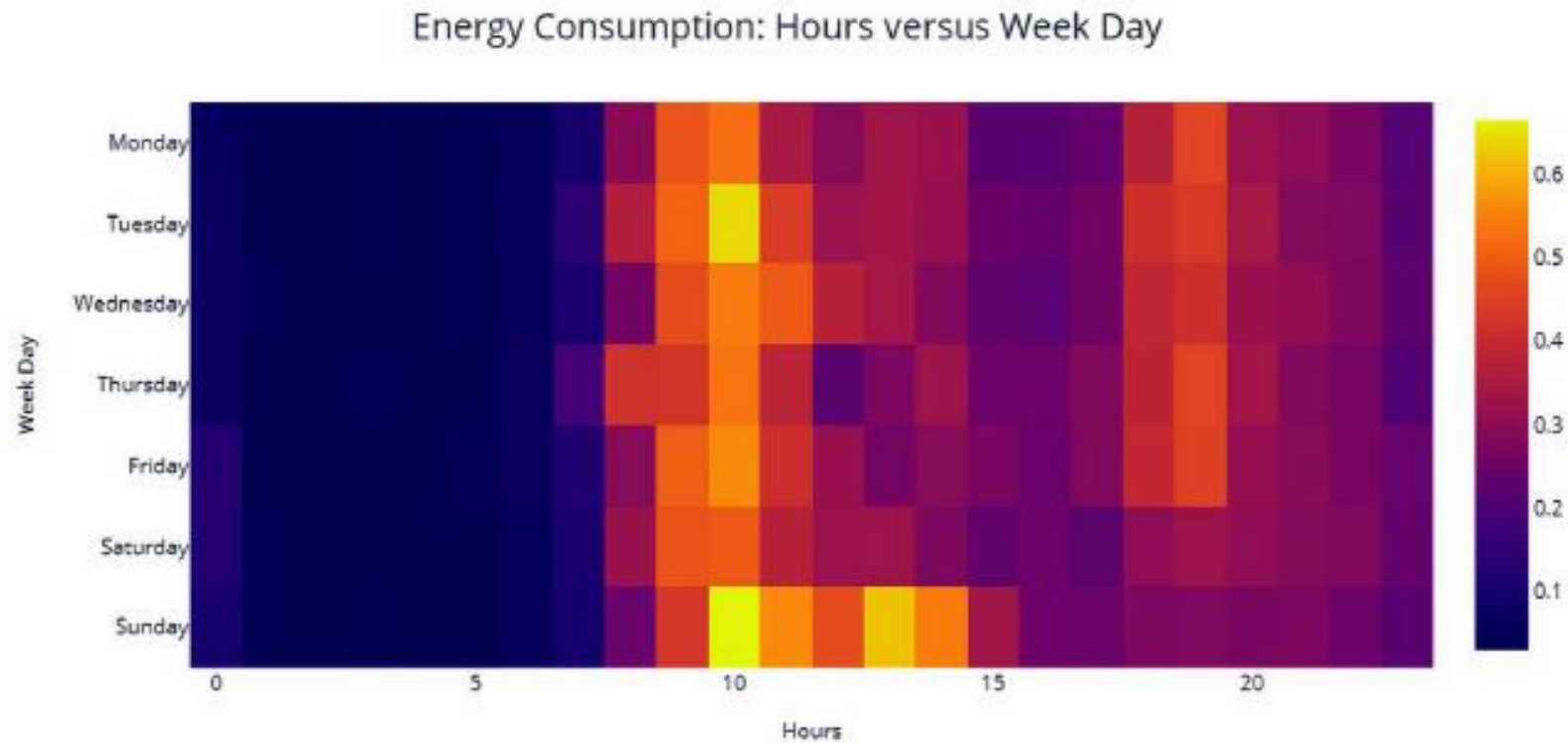


Figure 3.9: A calendar heatmap for energy consumption

Autocorrelation plot

Now, let's see what these plots look like for our household from the dataset (after making it stationary):

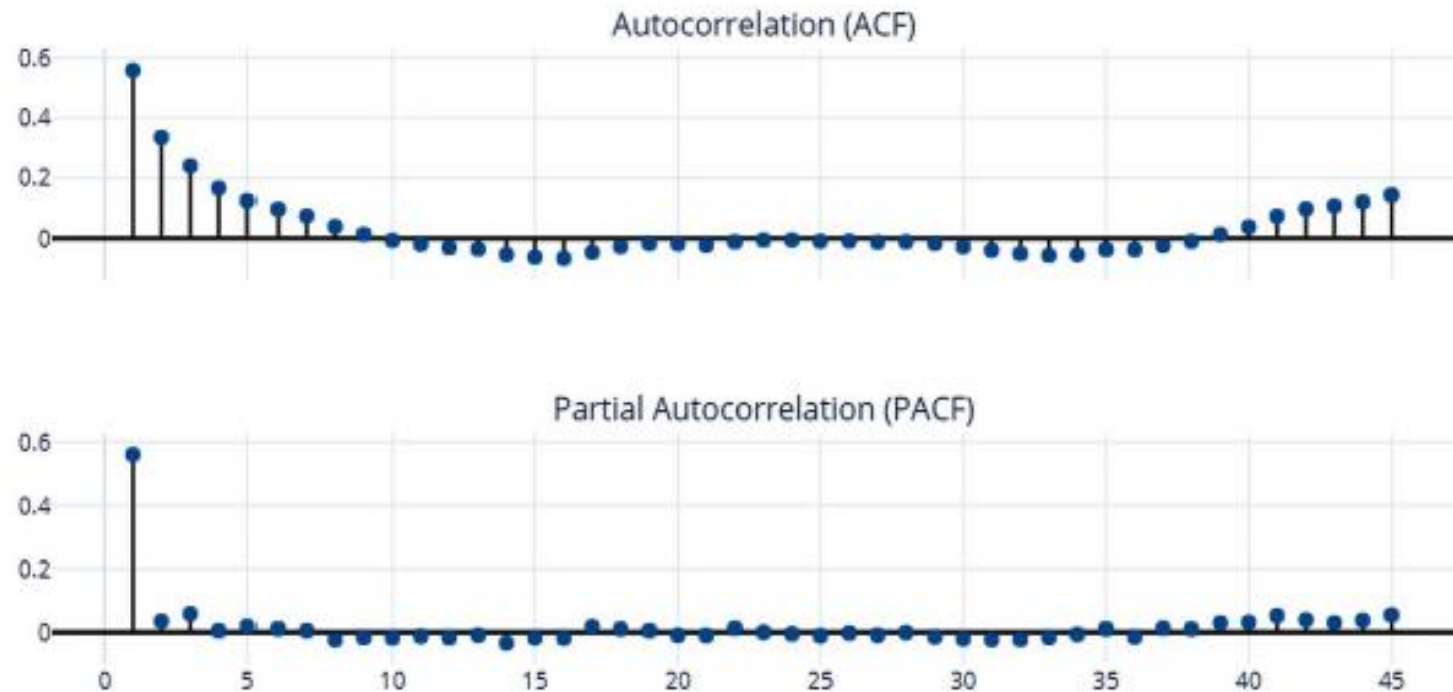


Figure 3.10: Autocorrelation and partial autocorrelation plots

Decomposing a time series

Seasonal decomposition is the **process by which we deconstruct a time series into its components** typically, **trend**, **seasonality**, and **residuals**.

The general approach for decomposing a time series is as follows:

1. **Detrending**: Here, we estimate the trend component (which is the smooth change in the time series) and remove it from the time series, giving us a detrended time series.
2. **Deseasonalizing**: Here, we estimate the seasonality component from the detrended time series. After removing the seasonal component, what is left is the residual.

Detrending

Detrending can be done in a few different ways.

Two popular ways of doing it are by using **moving averages** and **locally estimated scatterplot smoothing (LOESS) regression**.

Moving averages

Moving average along the time series can be seen as a window that is moved along the time series in steps, and at each step, the average of all the values in the window is recorded.

It helps us **estimate** the **slow change** in a time series, which is the trend. The downside is that the technique is quite **noisy**. Even after smoothing out a time series using this technique, the extracted trend will not be smooth; it will be noisy. The noise should ideally reside with the residuals and not the trend.

Detrending

LOESS

The LOESS (*locally weighted polynomial regression*) algorithm is a non-parametric method that is used to **fit a smooth curve onto a noisy signal**.

We use an **ordinal variable** that moves between the time series as the **independent** variable and the **time series** signal as the **dependent** variable.

For each value in the ordinal variable, the algorithm uses a **fraction of the closest points** and **estimates a smoothed trend** using only those points in a **weighted regression**. The weights in the weighted regression are the closest points to the point in question.

This is given the highest weight and it decays as we move farther away from it. This gives us a very effective tool for modeling the smooth changes in the time series (trend).

Deseasonalizing

The seasonality component can also be estimated in a few different ways.

The two most popular ways of doing this are by using **period-adjusted averages** or a **Fourier series**.

Period adjusted averages

Is a pretty simple technique wherein we calculate a seasonality index for each period in the expected cycle by taking the average values of all such periods over all the cycles.

For example, in monthly data with yearly seasonality, the cycle length is 12 months. We calculate the average value for each month across all years, such as the average of all January values, all February values, and so on.

This gives 12 monthly averages. Then we compare each monthly average to the overall average:

Additive seasonality: subtract the overall average.

Multiplicative seasonality: divide by the overall average.

The result is a seasonal index that shows how much each month is typically above or below the average.

Fourier series

A **Fourier series** represents a periodic pattern as a combination of sine and cosine waves. In time series, this is useful because **seasonality is periodic**.

Therefore, we can model seasonal behavior using a small number of Fourier terms.

The cycle length is denoted by **P**.

For example, in monthly data with yearly seasonality, **P = 12**.

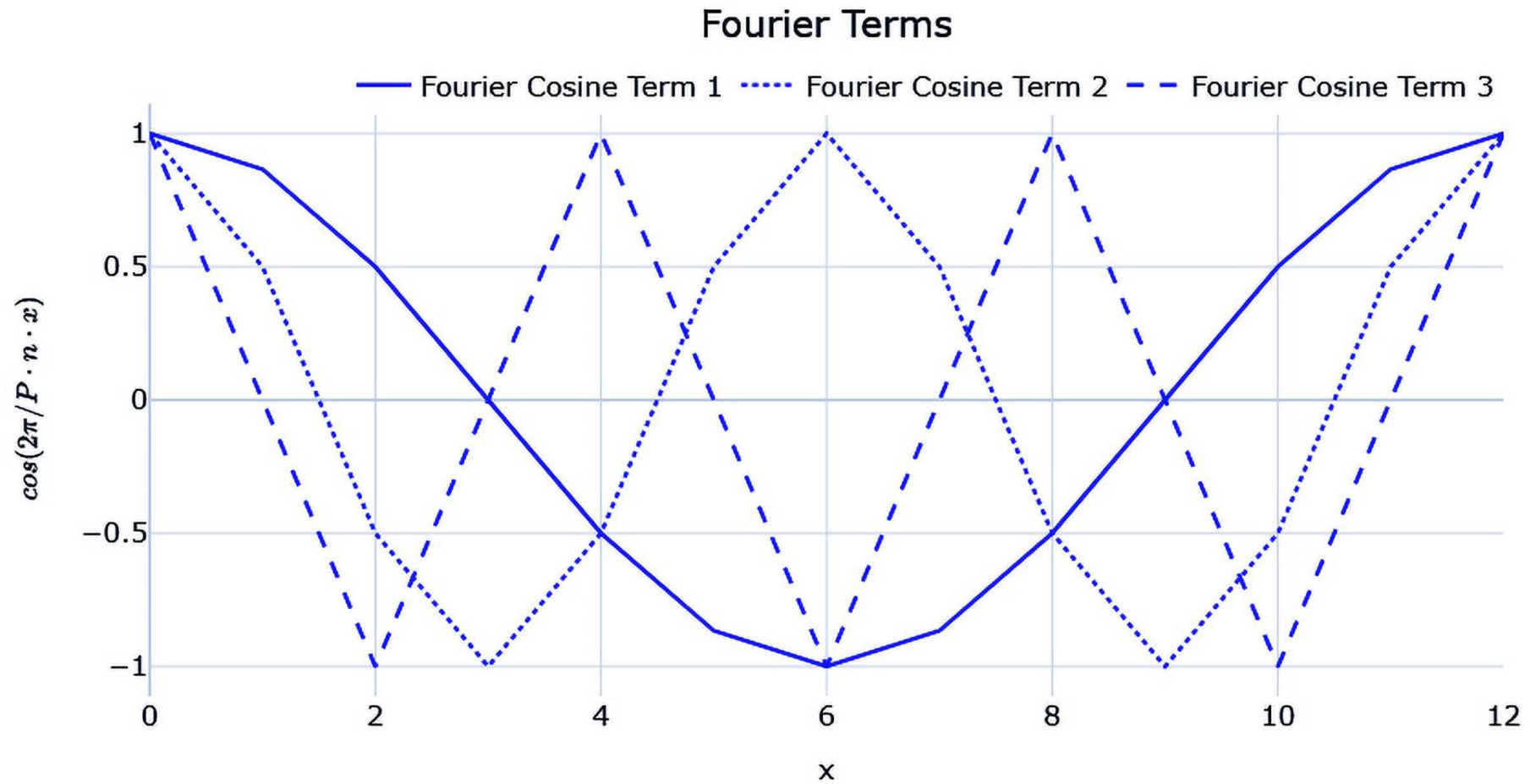
The number of Fourier terms, **N**, controls how complex the seasonal pattern can be:
Small **N** → simple, smooth seasonality.

Large **N** → more complex seasonality, but higher risk of overfitting.

So, Fourier terms help us capture seasonal patterns in a flexible but controlled way.

$$S_{N(x)} = \frac{a_0}{2} + \sum_{n=1}^N \left(a_n \cdot \cos\left(\frac{2\pi}{P} \cdot n \cdot x\right) + b_n \cdot \sin\left(\frac{2\pi}{P} \cdot n \cdot x\right) \right)$$

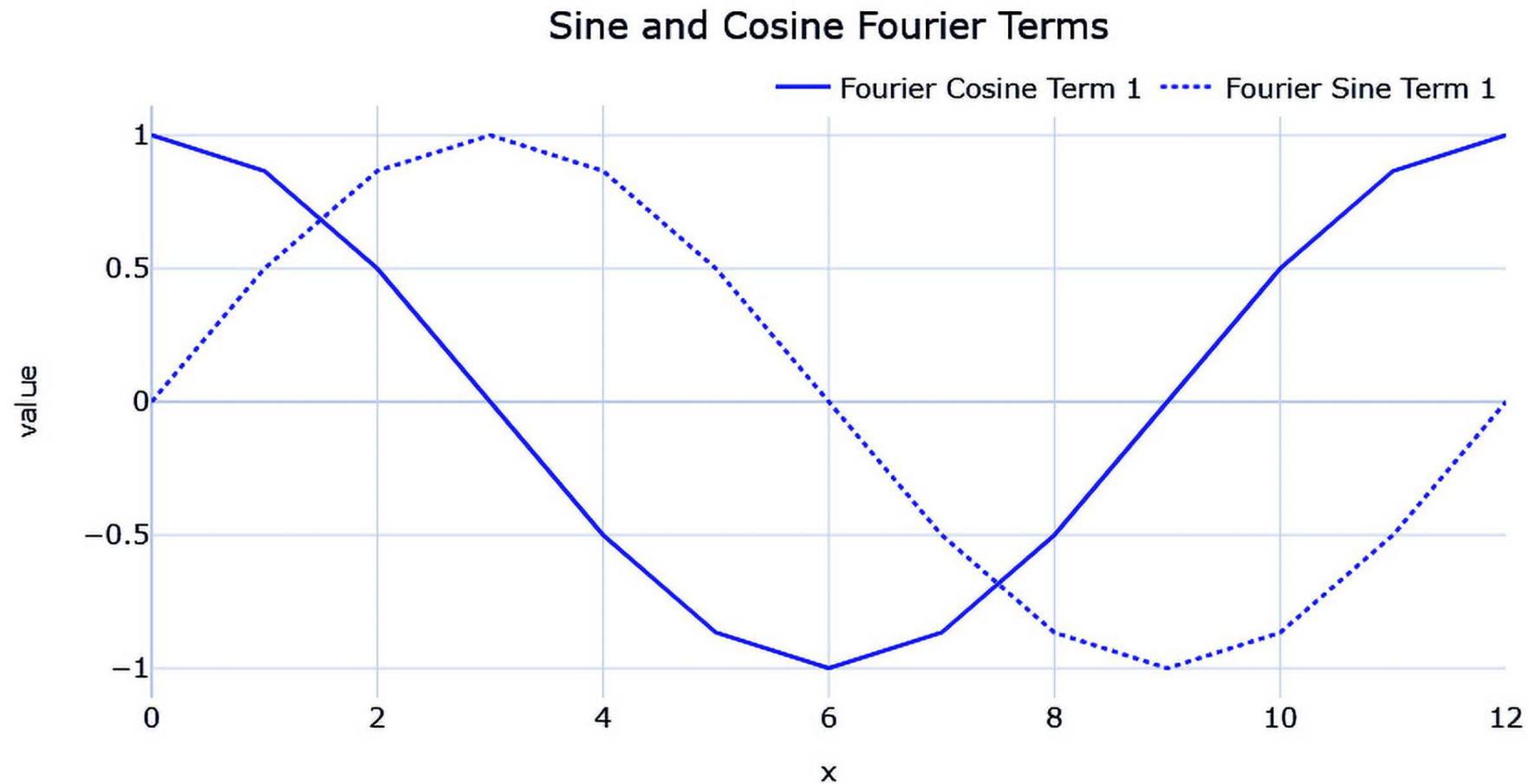
Fourier series



Cosine Fourier terms ($n = 1, 2, 3$)

Fourier series

The sine and cosine waves are complementary to each other



seasonal_decompose from statsmodel

seasonal_decompose from statsmodels.tsa.seasonal is used to split a time series into:

- **Trend**
- **Seasonality**
- **Residual/noise**

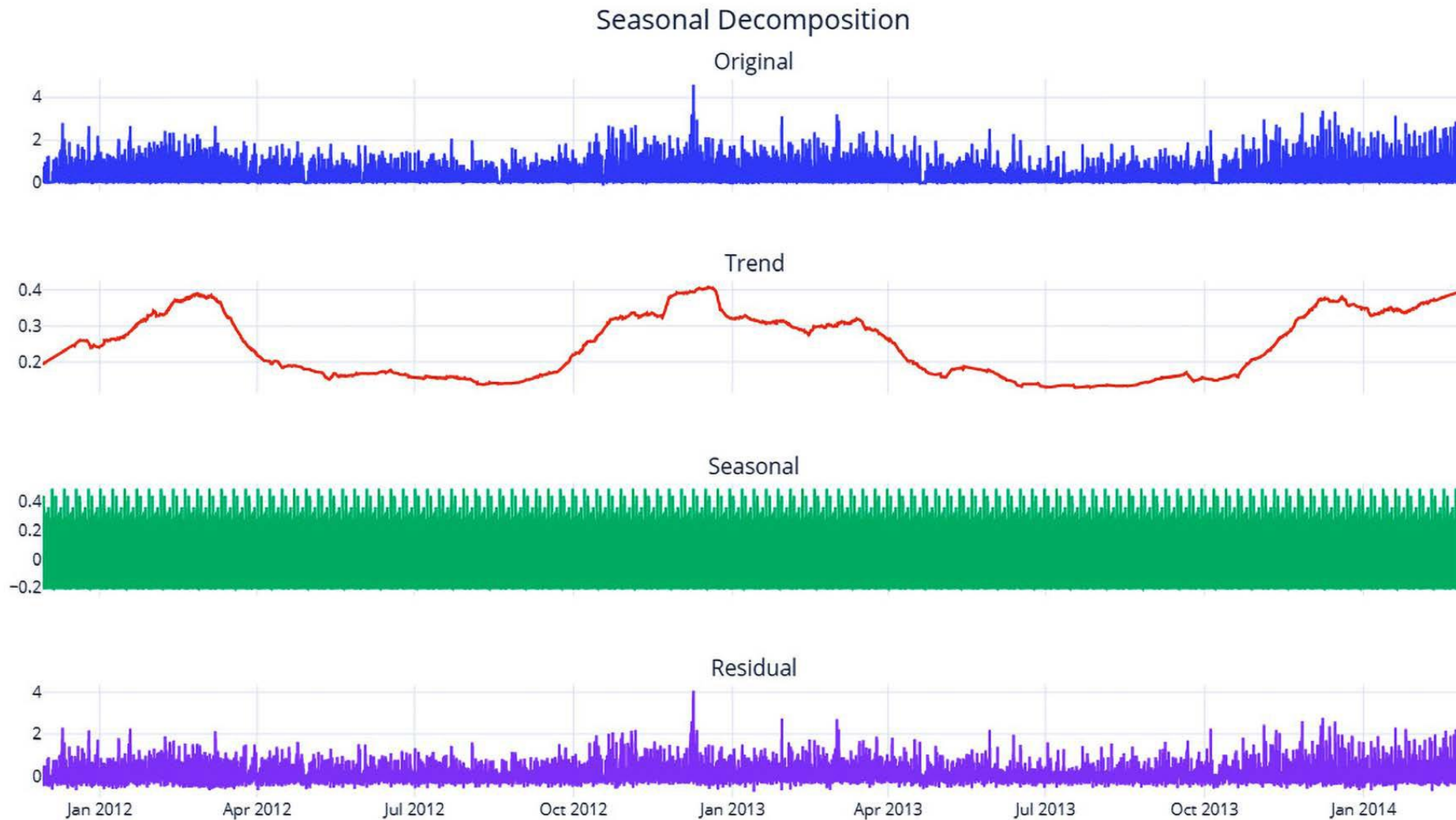
It estimates the trend using **moving averages** and the seasonal pattern using **period-based averages**.

Main parameters:

- **period**: length of the seasonal cycle.
- **model**: "additive" or "multiplicative".
- **filt**: controls the moving-average window.
- **extrapolate_trend**: fills trend values at the edges.
- **two_sided**: if True, uses past and future values; if False, uses only past values.

Important note!! seasonal_decompose does **not support missing values**, so missing data should be handled first.

seasonal_decompose from statsmodel



Seasonality and trend decomposition using LOESS (STL)

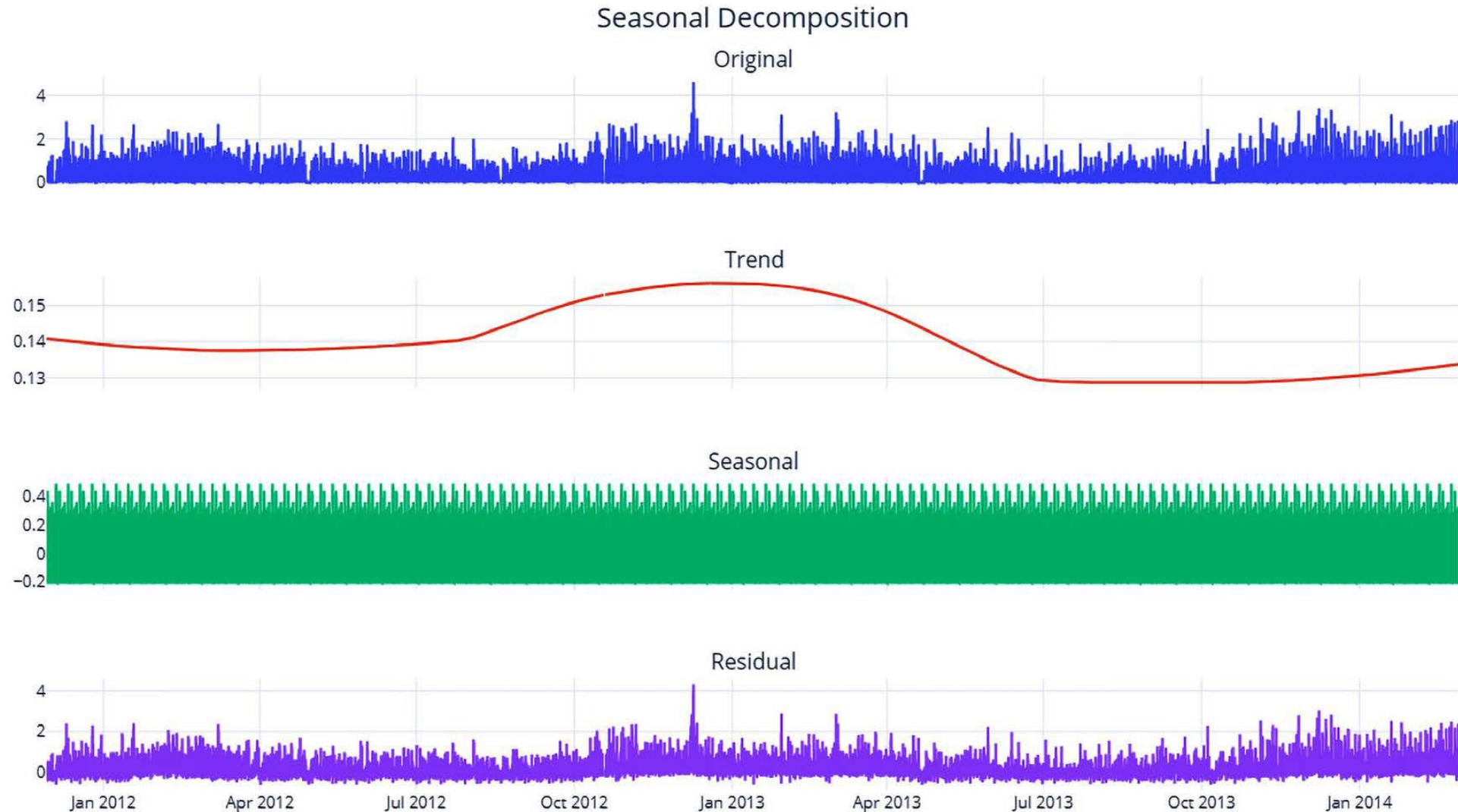
STL decomposition is used to **split a time series into trend, seasonality, and residuals**.

Unlike `seasonal_decompose`, STL uses LOESS to estimate the trend, making it **more flexible for complex trends**. Seasonality is estimated using period-based averages.

Main parameters:

- **seasonality_period**: length of the repeating seasonal cycle.
- **model**: additive or multiplicative decomposition.
- **lo_frac**: fraction of data used in LOESS trend fitting.
- **lo_delta**: speeds up computation by using interpolation instead of full regression.

Seasonality and trend decomposition using LOESS (STL)



Seasonality and trend decomposition using LOESS (STL)

Let's also look at the decomposition for just one month to see the extracted seasonality patterns clearer:

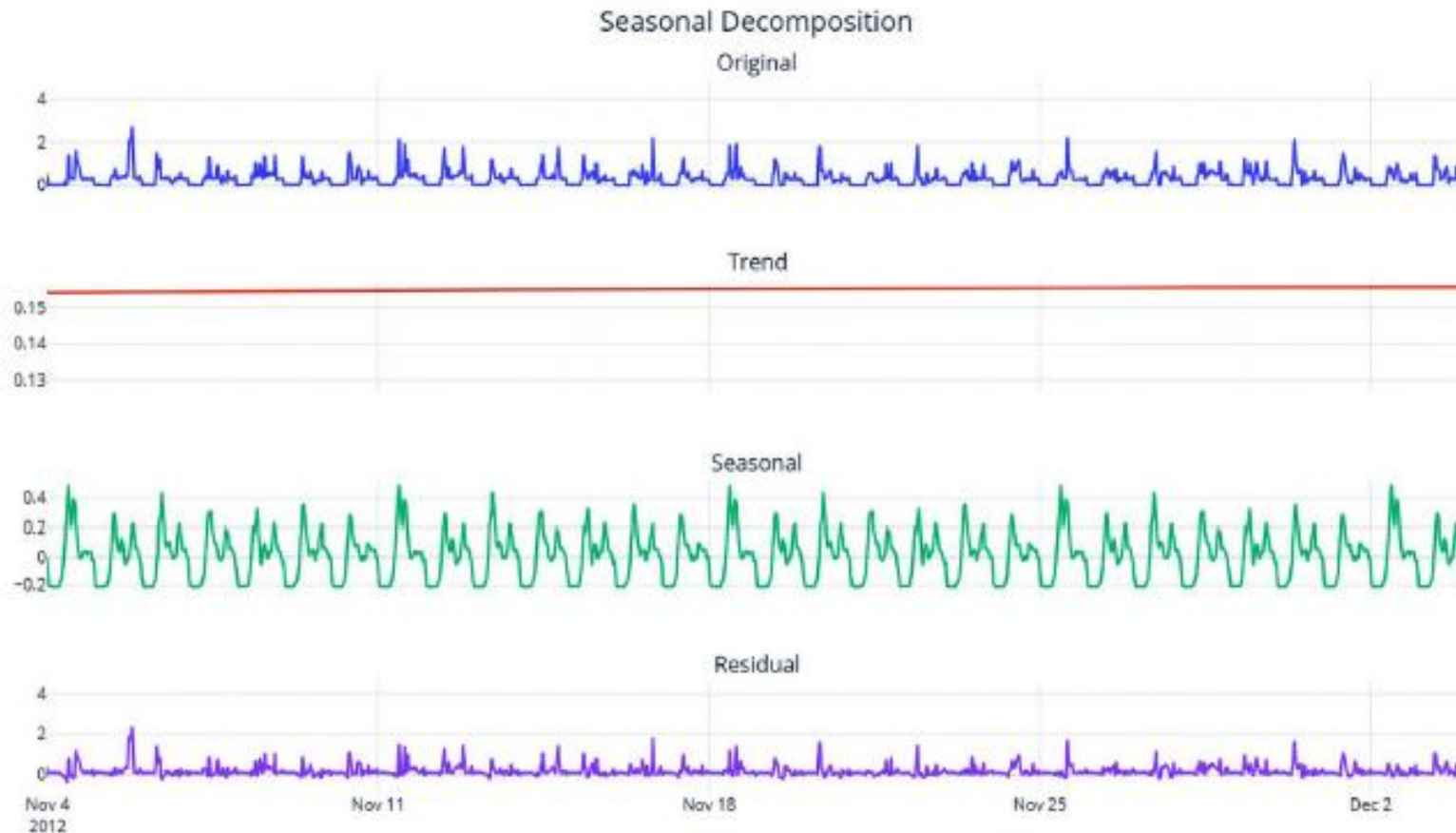


Figure 3.15: STL decomposition (zoomed-in for a month)

Fourier decomposition

```
stl = FourierDecomposition(seasonality_period="hour", model = "additive", n_
fourier_terms=5)
res_new = stl.fit(pd.Series(ts.squeeze(), index=ts_df.index))
```

We can find the Python implementation for decomposing a time series using Fourier terms in src.

Alternatively, we can create any custom seasonality array that's the same length as the time series that has an ordinal representation of the seasonality. If it is an annual seasonality of daily data, the array would have a minimum value of 1 and a maximum value of 365 as it increases by one every day of the year:

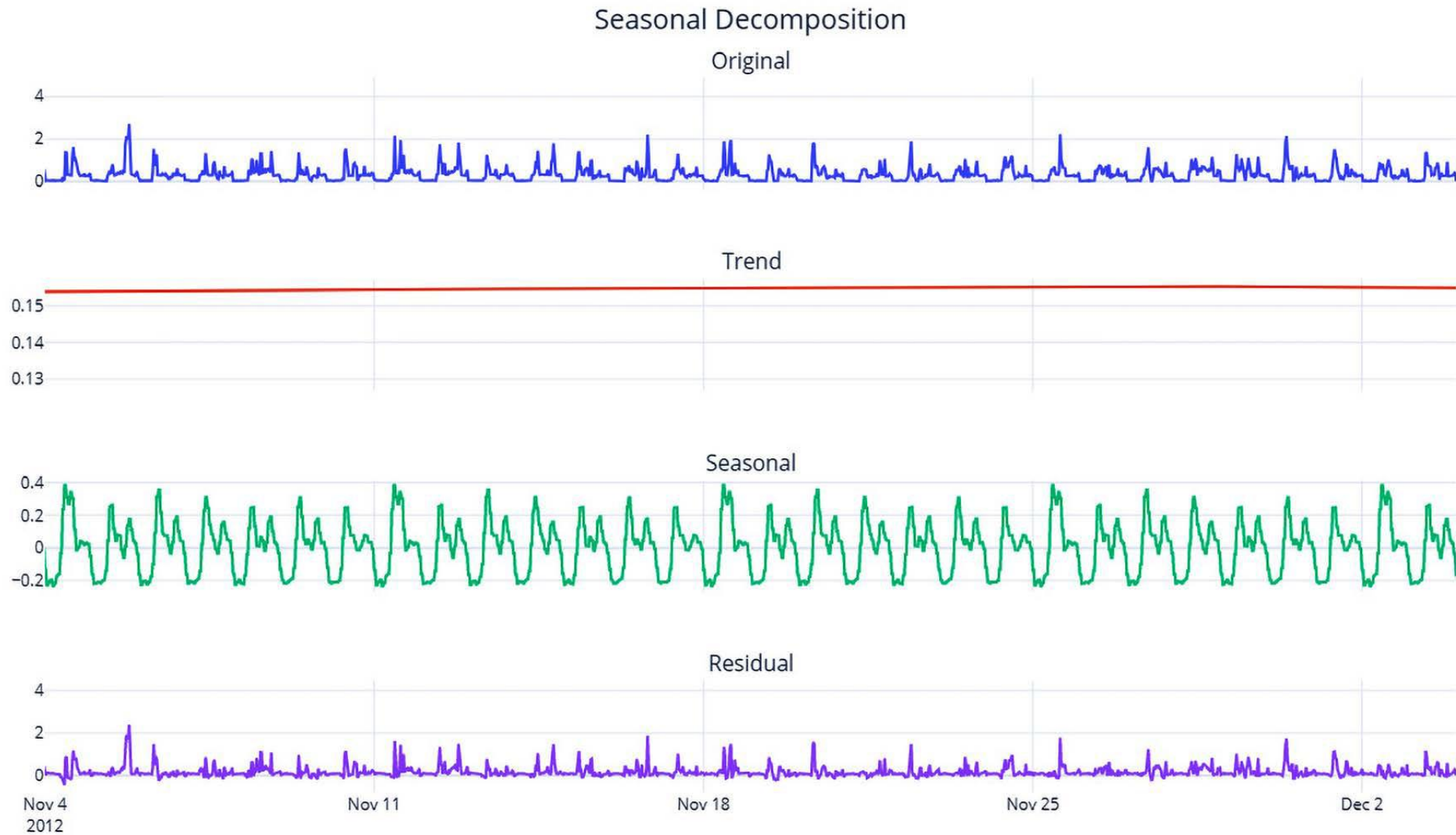
```
#Making a custom seasonality term
ts_df["dayofweek"] = ts_df.index.dayofweek
ts_df["hour"] = ts_df.index.hour
#Creating a sorted unique combination df
map_df = ts_df[["dayofweek", "hour"]].drop_duplicates().sort_
values(["dayofweek", "hour"])
# Assigning an ordinal variable to capture the order
map_df["map"] = np.arange(1, len(map_df)+1)
# mapping the ordinal mapping back to the original df and getting the
seasonality array
seasonality = ts_df.merge(map_df, on=["dayofweek", "hour"], how='left',
validate="many_to_one")['map']
stl = FourierDecomposition(model = "additive", n_fourier_terms=50)
res_new = stl.fit(pd.Series(ts, index=ts_df.index), seasonality=seasonality)
```

Fourier decomposition

The key parameters that are involved in this process are as follows:

- **seasonality_period** is the seasonality to be extracted from the *datetime index*. pandas datetime properties, such as `week_of_day` and `month`, can be used to specify the most prominent seasonality. If left set to `None`, you need to provide the seasonality array while calling `fit`.
- **model** takes `additive` or `multiplicative` as an argument to determine the type of decomposition.
- **n_fourier_terms** determines the number of Fourier terms to be used to extract the seasonality. The more we increase this parameter, the more complex the seasonality that is extracted from the data.
- **lo_frac** is the fraction of the data that will be used to fit the LOESS regression.
- **lo_delta** is the fractional distance within which we use linear interpolation instead of weighted regression. Using a non-zero `lo_delta` significantly decreases computation time.

Fourier decomposition



Multiple seasonality decomposition using LOESS (MSTL)

High-frequency time series, such as hourly or daily data, may contain **multiple seasonal patterns** at the same time, for example hourly, weekly, and yearly seasonality.

MSTL extends STL decomposition to handle several seasonalities instead of only one. This gives a better decomposition because remaining seasonal patterns are not incorrectly left inside the residuals.

Main parameters:

- **seasonality_periods**: list of seasonal cycles to extract.
- **seasonality_model**: uses either period averages or Fourier terms.
- **model**: additive or multiplicative decomposition.
- **n_fourier_terms**: controls the complexity of Fourier-based seasonality.
- **lo_frac**: fraction of data used for LOESS trend fitting.
- **lo_delta**: speeds up LOESS using interpolation.

```
stl = MultiSeasonalDecomposition(seasonal_model="fourier", seasonality_
periods=["day_of_year", "day_of_week", "hour"], model = "additive", n_fourier_
terms=10)
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

Multiple seasonality decomposition using LOESS (MSTL)

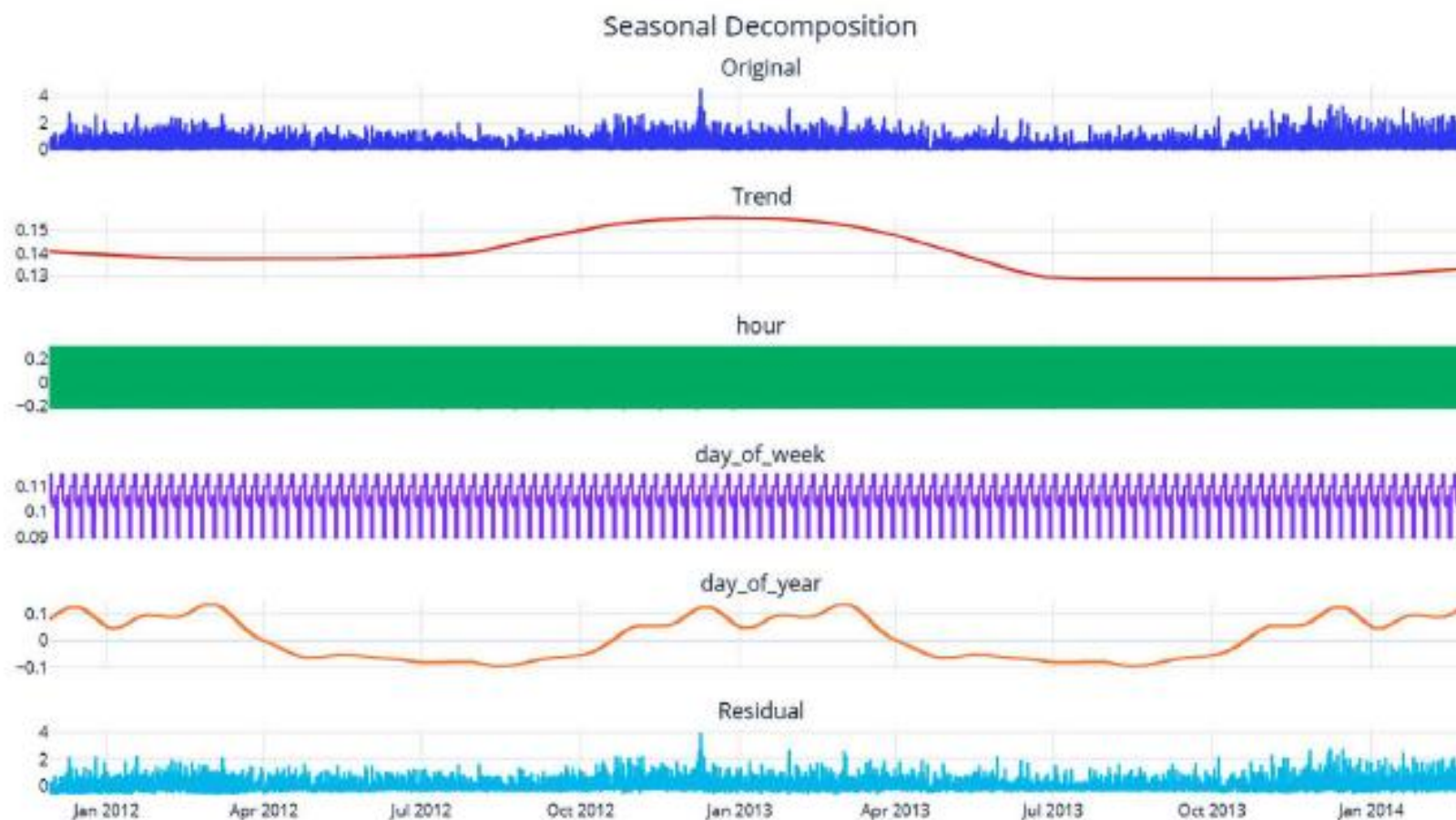


Figure 3.17: Multiple seasonality decomposition using Fourier terms

Multiple seasonality decomposition using LOESS (MSTL)

Here, we can see that the `day_of_week` seasonality has been extracted. To see the `day_of_week` and hour seasonal components, we need to zoom in a bit:

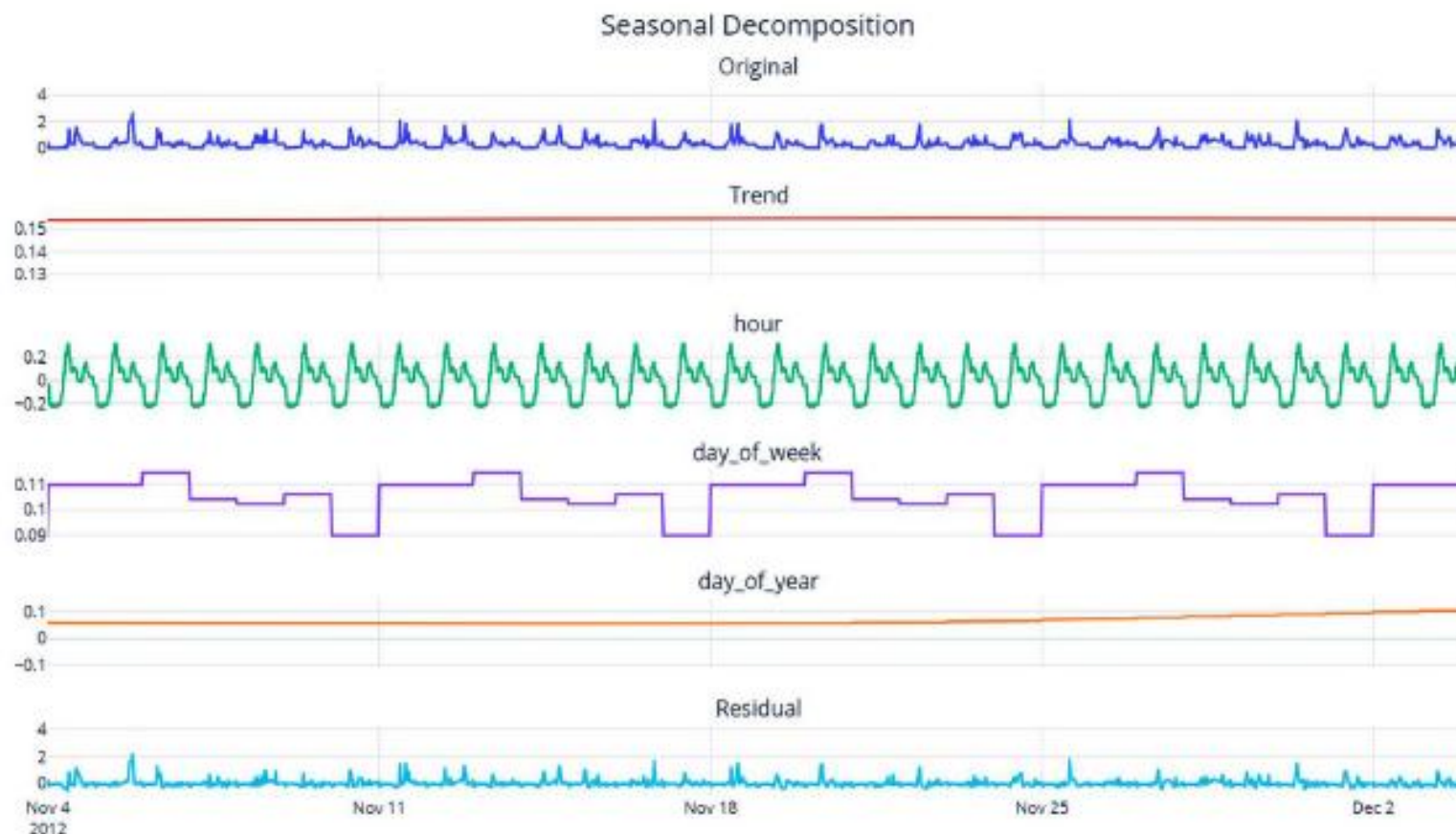


Figure 3.18: Multiple seasonality decomposition using Fourier terms (zoomed-in for a month)

The four techniques

Implementation	Trend	Seasonal	Supports Multiple Seasonality?	Supports Missing?
Seasonality decomposition	Moving Averages	Period-Adjusted Averages	No	No
STL	LOESS	Period-Adjusted Averages	No	Yes
Fourier decomposition	LOESS	Fourier Terms	No	No
Multiple seasonality decomposition	LOESS	Period-Adjusted Averages / Fourier Terms	Yes	No

Table 3.1: Different seasonal decomposition techniques